Highlights

A Multi-Objective Evolutionary Neural Architecture Search Framework for Text Generation

Israel Sánchez Miranda, Sergio Sarmiento-Rosales, Miriam Pescador Rojas, Víctor Adrián Sosa Hernández

- We propose a multi-objective evolutionary Neural Architecture Search framework for text generation based on a DistilGPT2-inspired design space.
- We define a continuous search space tailored to text generation, enabling the exploration of architectures with different trade-offs between semantic quality and model complexity.
- We incorporate a model-based performance predictor to make the search computationally feasible under constrained training budgets.
- The recovered architectures improve METEOR over the selected DistilGPT2 baseline and reveal distinct trade-offs between semantic quality and hardware-oriented efficiency.

A Multi-Objective Evolutionary Neural Architecture Search Framework for Text Generation

Israel Sánchez Miranda[a], Sergio Sarmiento-Rosales[a], Miriam Pescador Rojas[b], Víctor Adrián Sosa Hernández[a]

[a] *Tecnologico de Monterrey, School of Engineering and Sciences*
[b] *Instituto Politécnico Nacional, Escuela Superior de Cómputo*

Abstract

Recent advances in Natural Language Processing (NLP), have produced increasingly capable models, but often at the expense of high computational cost and labor-intensive manual architecture design. These limitations reduce scalability, accessibility, and deployment in resource-constrained environments. To address this, we propose a Neural Architecture Search (NAS) framework for the automated design of text generation architectures that balance semantic quality and efficiency. The framework defines a search space of deep neural network (DNN) architectures based on DistilGPT2 and explores it using two multi-objective NAS strategies: SMS-EMOA and Lex-MODES. The search is formulated as a bi-objective optimization problem, where quality is measured through METEOR score and efficiency through parameter count. To keep the process computationally feasible, a model-based predictor estimates architecture performance without fully training every candidate. Results show that the discovered architectures outperform the DistilGPT2 baseline, reaching METEOR values up to 0.72, while also exhibiting distinct hardware-oriented profiles with reduced deployment cost. Overall, the framework demonstrates that NAS can identify useful trade-offs between text generation quality and practical efficiency, enabling the design of more accessible and deployable NLP architectures.

Keywords: Text Generation, DistilGPT2, Neural Architecture Search, Multi-Objective Optimization

Email addresses: a01378705@tec.mx (Israel Sánchez Miranda), A01801059@tec.mx (Sergio Sarmiento-Rosales), mpescadorr@ipn.mx (Miriam Pescador Rojas), vsosa@tec.mx (Víctor Adrián Sosa Hernández)

1. Introduction

Natural language processing (NLP) is playing an increasingly central role in modern applications, including keyword extraction, automated annotation, text generation, spelling correction, and question-answering systems [1, 2, 3, 4, 5]. Within this landscape, text generation has emerged as one of the most rapidly evolving areas, driven by the demand for more capable, responsive, and sophisticated generative models. However, these advances have come at the cost of increasing computational requirements. As a result, text generation models often remain difficult to scale and deploy, particularly in resource-constrained environments where efficiency is as important as predictive quality.

Given the high computational demands of text generation architectures, improving their performance in resource-constrained environments is particularly important. This challenge is amplified by the increasing prevalence of edge and mobile devices, where architectures must satisfy not only quality requirements but also deployment constraints [6, 7].

Despite these demands, the design of text-generation deep neural networks (DNNs) still relies heavily on domain expertise and manual architecture engineering. As model complexity continues to increase, this handcrafted process becomes slower, harder to scale, and less likely to identify truly effective architectures. Consequently, automating neural architecture design emerges as a natural and necessary step toward the development of more competitive and efficient text-generation models [8].

Neural Architecture Search (NAS) is an automated framework for exploring and optimizing neural network architectures. It has demonstrated strong performance across multiple domains, including recent applications to NLP and text generation [9, 10, 11]. Following Elksen et al. [12], NAS is commonly described through three main components: the search space, which defines the set of candidate architectures; the search strategy, which governs how candidates are generated and selected; and the performance estimation strategy, which evaluates candidate quality. Since fully training every candidate is computationally impractical, efficient estimation mechanisms such as model-based predictors or zero-cost proxies play a central role in enabling large-scale exploration. Although NAS has shown considerable success in areas such as computer vision, its development in NLP remains comparatively

limited. In particular, despite recent advances in NAS for transformer-based models [13], identifying architectures that remain competitive while adapting to diverse hardware constraints is still an open challenge [14].

This work desings and adapts the main components of NAS in the context of text generation. First, we define a DistilGPT2-inspired search space tailored to this task. Second, we consider two evolutionary search strategies to explore the proposed space under semantic quality and efficiency criteria. Third, to make the search computationally feasible, we incorporate a model-based performance predictor that estimates candidate quality without requiring full training of every architecture. Under this formulation, the study investigates whether these elements can be integrated into a unified NAS framework for text generation and whether the resulting architectures can recover meaningful trade-offs between semantic quality and deployment-oriented efficiency. In this way, the proposed approach may support the development of more accessible and resource-aware text generation architectures for constrained environments.

The remainder of this manuscript is organized as follows. Section 2 presents the theoretical background on text generation, NAS, and hardware-aware optimization. Section 3 describes the proposed framework, including the search space, the search strategies, and the model-based evaluation stage. Section 4 presents and discusses the experimental results. Finally, Section 5 summarizes the main conclusions and outlines directions for future research.

2. Background and Related Work

This section reviews the main challenges associated with existing text-generation architectures and summarizes the state of the art in NAS and hardware-aware NAS for this task. In addition, it highlights the methodological components most relevant to the present study, including search-space design, search strategies, and efficient evaluation approaches.

2.1. Neural Architecture Search

NAS is an automated framework for discovering DNN architectures under predefined design constraints and optimization objectives [12]. A standard NAS pipeline is typically described through three main components: the search space, which defines the candidate architectures; the search strategy, which governs how these candidates are generated and selected; and the evaluation mechanism, which estimates their quality [12].

Despite its success across multiple domains, NAS often remains computationally expensive because it requires the repeated evaluation of many candidates, especially when full training is involved [15]. To alleviate this limitation, several efficient alternatives have been proposed, including weight sharing, low-fidelity approximations, zero-cost proxies, and performance predictors [15, 12]. Among these, model-based NAS is particularly attractive in expensive search settings, since it enables candidate evaluation without exhaustive training.

In many practical scenarios, however, predictive quality alone is not sufficient, as candidate architectures must also satisfy deployment-related constraints such as parameter count, latency, memory usage, or energy consumption. This has motivated the development of multi-objective and hardware-aware NAS approaches, which aim to identify architectures that remain competitive while meeting the requirements of target devices [14].

2.2. Neural Architecture Search for Text Generation

Recent research at the intersection of text generation and NAS has emphasized the need to adapt search methods to real computational constraints [14]. In particular, the growing computational and energy demands of modern text-generation models remain a major barrier to accessibility and practical deployment. As noted by Kinde [16], text generation is typically a resource-intensive task that often depends on specialized hardware, which limits the broader democratization of NLP technologies. In this context, hardware-aware NAS has emerged as a relevant direction for integrating efficiency considerations into the search process, with the goal of recovering architectures that are not only competitive in quality but also more suitable for real deployment scenarios.

At the same time, NAS itself remains a challenging optimization problem. Ren et al. [17] highlight the risk of premature convergence caused by the nonconvexity of the search space, which makes effective exploration a central issue. In this respect, bioinspired and evolutionary methods are particularly attractive because their population-based search dynamics can help mitigate convergence to local optima [18]. More broadly, the computational complexity of NAS is shaped by factors such as dataset size, input dimensionality, and the structure of the search space, making task-specific NAS design both expensive and methodologically demanding [12].

Existing NAS approaches for text generation can be broadly divided into task-specific NLP formulations and more general hardware-aware methods.

In addition, several complementary strategies have been explored to improve search efficiency and deployment suitability. These include evolutionary formulations with parameter sharing among child architectures [19], knowledge distillation to transfer capability from larger models to lightweight alternatives [20, 21], and task-dependent linguistic constraints that reduce generation complexity while preserving coherence [22]. Together, these developments highlight both the promise and the current methodological fragmentation of NAS for text generation.

2.3. Multi-objective and Hardware-Aware NAS

In many practical deployment scenarios, optimizing neural architectures solely for predictive quality is insufficient, since real-world applications must also satisfy constraints such as parameter count, latency, memory usage, and energy consumption. This has motivated the development of multi-objective NAS (MoNAS), where the goal is to recover a set of trade-off architectures rather than a single optimum [12]. Representative examples include NSGA-Net [23], which formulated NAS as a multi-objective evolutionary optimization problem, and DPP-Net [24], which incorporated both device-agnostic and device-related objectives to recover Pareto-optimal architectures across multiple hardware platforms.

This perspective naturally leads to hardware-aware NAS, where hardware-dependent measures are considered during search instead of relying only on architecture-agnostic proxies such as FLOPs. Representative approaches include MnasNet [25], which introduced platform-aware NAS for mobile devices, ProxylessNAS [26], which enabled direct search on target hardware, and FBNet [27], which combined differentiable NAS with latency-aware optimization. More recent work, such as MO-HDNAS [28], further highlights the importance of scalable and explicitly multi-objective HWNAS for recovering efficient architectures under diverse deployment constraints. Together, these developments make multi-objective and hardware-aware NAS particularly relevant for resource-constrained settings, where predictive quality must be balanced against deployment efficiency.

3. Text Generation NAS Framework

This section describes the proposed NAS framework for text generation, including the search-space design, the bi-objective formulation, the search

strategies considered, and the model-based evaluation mechanism used to efficiently recover architectures under semantic quality and efficiency criteria.

3.1. Search Space

The proposed search space is built upon DistilGPT2, a knowledge-distilled version of GPT2 selected as the baseline architecture due to its favorable balance between generation capability and computational efficiency [29, 30]. As illustrated in Figure 1, the architecture is composed of four main components: text embedding, positional embedding, a stack of *Transformer units*, and a final normalization layer. Each *Transformer unit* contains an attention cell followed by a fully connected cell. Within the proposed search space, only the components located between the positional embedding and the final normalization layer are optimized, while the embedding and normalization blocks are kept fixed.

Candidate architectures are represented through a block-parameter-based search space using a mixed integer-real encoding, where each chromosome defines the structural and functional configuration of the searchable Transformer stack. The representation is defined as follows:

$$N = [e, l, n, h, r, a, g], \tag{1}$$

where N denotes a candidate architecture, e is the embedding dimension, l is the maximum embedding sequence length, n is the number of *Transformer units*, h is the number of attention heads, r is the residual dropout, a is the attention dropout, and g is the activation function used in the feed-forward layers. The admissible values associated with each design variable are summarized in Table 1. For continuous optimization, the categorical activation function g is mapped to the interval $[0, 1]$, using the subranges GeLU $\in [0.0, 0.2)$, ReLU $\in [0.2, 0.4)$, Tanh $\in [0.4, 0.6)$, Swish $\in [0.6, 0.8)$, and Sigmoid $\in [0.8, 1.0]$. Although the search space is continuous due to the real-valued variables, its size was approximated as $\sim 1.008 \times 10^7$ by discretizing the continuous dimensions into 20 bins to delimit search space size.

3.2. Search Strategy

The proposed search is formulated as a bi-objective optimization problem that seeks architectures with high semantic quality and low complexity. The objectives are to maximize METEOR and minimize parameter count. METEOR was selected because it captures semantic similarity through synonym

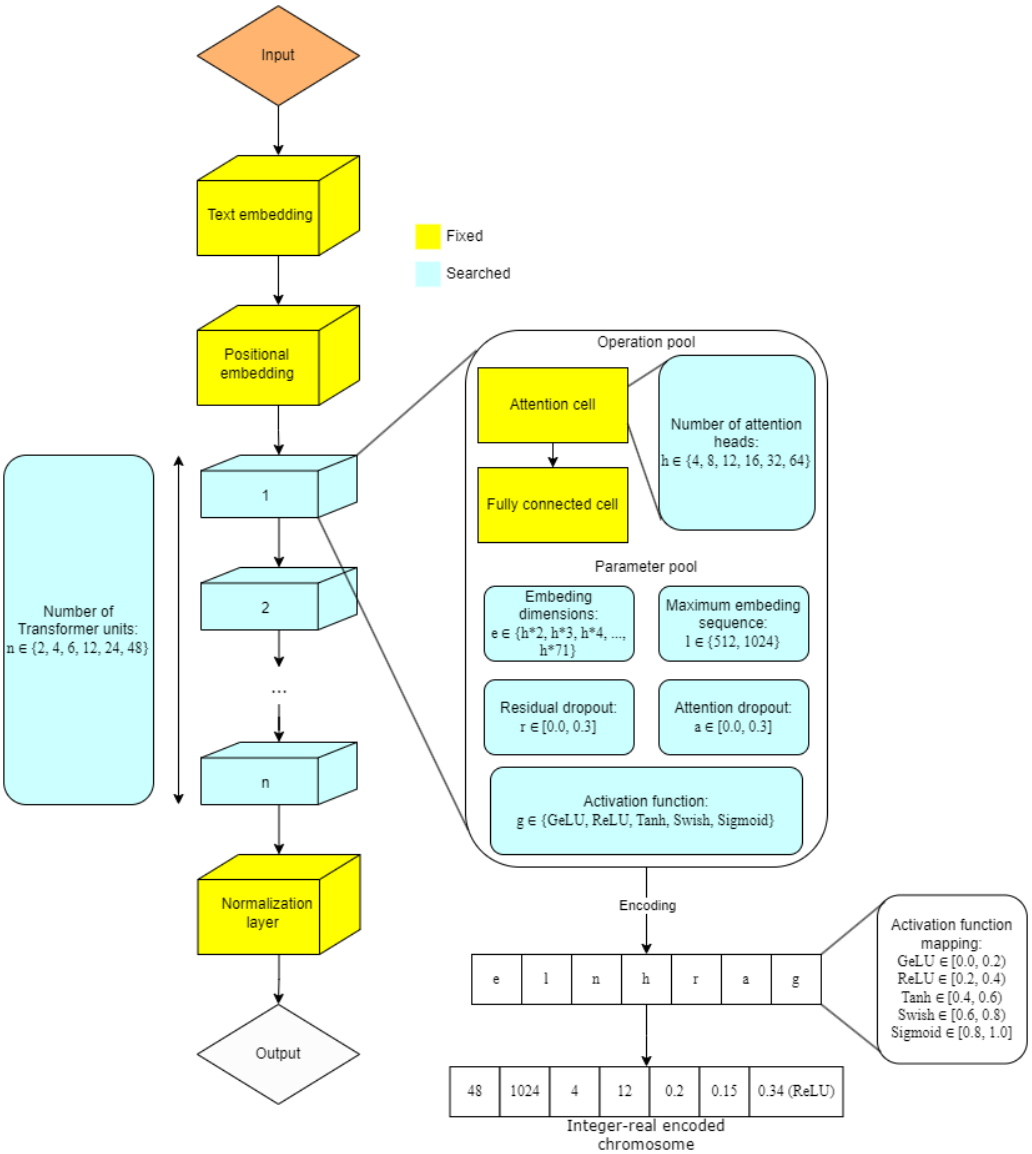

Figure 1: Tailored search space based on the DistilGPT2 architecture. Fixed components (yellow) include the text embedding, positional embedding, and final normalization layer. Searchable components (cyan) include the number of *Transformer units* (n), attention heads (h), embedding dimension (e), maximum sequence length (l), residual dropout (r), attention dropout (a), and activation function (g). The bottom part illustrates the mixed integer-real chromosome representation used during optimization.

Table 1: Search-space variables for the proposed DistilGPT2-based representation.

Var.	Description	Values / range
e	Embedding dimension	$\{h \times 1, h \times 2, \ldots, h \times 70\}$
l	Maximum sequence length	$\{512, 1024\}$
n	*Transformer units*	$\{2, 4, 6, 12, 24, 48\}$
h	Attention heads	$\{4, 8, 12, 16, 32, 64\}$
r	Residual dropout	$[0.0, 0.3]$
a	Attention dropout	$[0.0, 0.3]$
g	FFN activation	$\{\text{GeLU}, \text{ReLU}, \text{Tanh}, \text{Swish}, \text{Sigmoid}\}$

matching and stemming, often aligning more closely with human judgment in text generation tasks. Although parameter count is not a complete proxy for computational cost, as actual resource usage also depends on architectural organization, optimization dynamics, and hardware-specific effects [31, 32, 33], it remains a practical and interpretable efficiency objective in NAS [34, 35]. Two search strategies are considered in this study: SMS-EMOA as a reference multi-objective evolutionary optimizer and Lex-MODES as the main lexicographic search strategy.

3.2.1. SMS-EMOA

SMS-EMOA has shown strong performance in bi-objective optimization because it combines Pareto dominance with hypervolume contribution during survivor selection [36, 37, 38]. In the present implementation, a fixed-size population is evolved using simulated binary crossover (probability 0.9, $\eta = 15$) and polynomial mutation ($\eta = 20$), following the standard SMS-EMOA formulation.

Survivor selection is driven by hypervolume contribution with respect to the reference point $[1 \times 10^9, 0.0]$. The first component provides a conservative upper bound for parameter count, whereas the second corresponds to the minimum METEOR value. Since METEOR is a maximization objective, it is negated to comply with the minimization-based hypervolume computation.

3.2.2. Lex-MODES

To explore the proposed search space, the main optimizer adopts a lexicographic evolutionary formulation referred to as *Lexicographic Selection Multi-Objective Differential Evolution Strategy* (Lex-MODES). This method

combines a variant of the (μ+1) Evolutionary Strategy (ES), Differential Evolution (DE) mutation and crossover operators, and an SMS-EMOA-inspired survival mechanism [36]. This approach is designed to focus on parameter count as main objective since we plan to steer to a more hardware aware approach. Further, we expect that by generating this bias toward one objective arquitectures complement SMS-EMOA solution set.

At each generation, offspring are generated through DE variation and evaluated before selecting the next population from the union of parents and offspring. Unlike standard SMS-EMOA, survivor selection is not based on non-dominated sorting. Instead, candidate architectures are ranked lexicographically: parameter count is used as the primary criterion to prioritize efficiency, whereas METEOR is used to break ties. A fixed number of generations t is adopted as the stopping criterion, providing controlled termination and bounded search cost.

For variation, Lex-MODES employs the DE/best/1/bin scheme. Mutation generates a trial vector u_i from the current best parent vector $\hat{x}$ and two randomly selected population vectors r_2 and r_3 ($r_2 \neq r_3$), according to

$$u_i = \hat{x} + F \times (r_2 - r_3)\,. \tag{2}$$

Here, F is the scaling factor controlling the differential contribution of the sampled vectors. The resulting trial vector is then recombined with its corresponding parent vector x_i using binomial crossover to produce an offspring:

$$x'_{ij}(t) = \begin{cases} u_{ij}(t), & \text{if } j \in \mathcal{J} \\ x_{ij}(t), & \text{otherwise}\,, \end{cases} \tag{3}$$

where $\mathcal{J}$ denotes the crossover mask. Following the standard DE/bin formulation, at least one allele is always inherited from the trial vector to guarantee variation.

Because the proposed representation includes both discrete and continuous design variables, all alleles are mapped to the interval $[0, 1]$ according to their respective domains. This continuous encoding allows DE operators to be applied directly while preserving the structural constraints of the architecture representation [39, 40, 41, 42, 43, 44]. After variation, a repair mechanism is applied to restore validity and ensure that all offspring remain within the admissible search-space constraints.

To improve the balance between exploration and exploitation, the scaling factor F is self-adapted using the 1/5 success rule. Every n generations, F is updated according to the successful mutation ratio p_s:

$$F(t+1) = \begin{cases} F(t) \times c, & \text{if } ps < \frac{1}{5} \\ \frac{F(t)}{c}, & \text{if } ps \geq \frac{1}{5}\,, \end{cases} \tag{4}$$

where $c = 0.817$ regulates the mutation step size, following empirical recommendations in evolution strategy literature [45, 46]. This mechanism dynamically adjusts the mutation strength according to recent search success.

3.3. Model-based Evaluation

To make the proposed NAS framework computationally tractable, a model-based evaluation stage is incorporated. Given the size of the search space, exhaustive training and evaluation of all candidate architectures is infeasible. For instance, assuming an average training time of 20 minutes per architecture, full exploration would require nearly 140,000 days. To overcome this limitation, the proposed framework replaces full training during search with two efficient evaluation components.

First, a performance predictor is used to estimate the METEOR score directly from the encoded representation of each candidate architecture, thereby avoiding costly full training during the search process [47, 48]. Second, a deterministic decoding-based procedure is employed to estimate the number of trainable parameters after architectural reconstruction. Together, these components provide the objective values required by the search algorithm while substantially reducing the computational burden. Although the present implementation focuses on METEOR prediction and parameter estimation, the same formulation could be extended to alternative low-cost evaluation mechanisms.

3.3.1. Training Data and Predictor Candidates

The dataset used to train the performance predictor was generated by sampling 300 candidate architectures from the proposed search space and fully training them on WikiText2 [49] to obtain the corresponding METEOR scores. To improve coverage of the decision space, candidate architectures were selected through Latin hypercube sampling [50], and the resulting dataset was stored as input–output pairs of the form `(architecture, score)`.

We consider only four regressor models as performance predictors candidates: Support Vector Regressor (SVR), Random Forest Regressor (RFR), Extreme Gradient Boosting regressor (XGB), and Multi-Layer Perceptron regressor (MLP). Four regressors delimition obey that this models perform well in other analysis cite [48]. However is posible to incorporate other techiques following the same pipeline. Prior to training, the input representation was normalized to ensure comparable feature scales and reduce fitting bias. Each model was tuned using `GridSearch` with 3-fold cross-validation, where Mean Absolute Error (MAE) was used as the model-selection criterion. The selected configurations were then assessed through 10 repetitions of 5-fold cross-validation to obtain more robust estimates of predictive performance.

The evaluation indicators were MAE, Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), coefficient of determination (R^2), Pearson correlation (PC), and Spearman correlation (SC).

3.3.2. Performance Predictor Selection

A Bayesian statistical comparison was performed to identify the predictor most suitable for integration into the NAS loop. As shown in Table 2, the strongest evidence in the error-oriented criteria favored the MLP regressor. In particular, MLP obtained the highest posterior support in RMSE across all pairwise comparisons and also showed the best performance in MAPE. By contrast, XGB and RFR were more competitive in MAE, whereas all models remained statistically equivalent in MSE.

For model-based NAS, the key requirement is not only to preserve relative trends, but to provide sufficiently accurate estimates of the objective values that guide the search. From this perspective, the evidence supporting MLP in RMSE and MAPE is especially relevant. Accordingly, although some competing models remained competitive in secondary indicators, MLP offered the most appropriate overall trade-off for integration into the proposed framework and was therefore selected as the performance predictor.

3.4. Evaluation Pipeline

The proposed evaluation pipeline consists of two stages. First, the DistilGPT2 baseline is trained and evaluated on WikiText2 [49] to establish a reference point in terms of semantic quality and computational behavior.

Table 2: Bayesian comparison summary across the four performance predictors.

Performance measure	Best model	Bayesian evidence	Role
MAE	XGB/RFR	Both outperform SVR and MLP (1.0000)	Secondary
MSE	Tie	Full equivalence across all models (1.0000)	Neutral
RMSE	MLP	vs SVR: 0.9964, vs XGB: 0.9828, vs RFR: 0.9998	Primary
MAPE	MLP	vs SVR: 0.7929, vs XGB: 1.0000, vs RFR: 1.0000	Primary
R^2	MLP	Best reported model	Supportive
Pearson	MLP	Best reported model	Supportive
Spearman	MLP	0.9970, 1.0000, 0.9993	Supportive
Overall	MLP	Strongest evidence in accuracy-oriented criteria	Selected

Second, the architectures obtained through NAS are evaluated under the same protocol, enabling direct comparison against the baseline.

This design ensures methodological consistency across all evaluated models. Therefore, performance differences can be attributed to the architectural changes introduced by the search process rather than to variations in training or inference conditions.

3.4.1. METEOR Score and Hardware-aware Measures

To assess post-search performance, the evaluation pipeline focuses on the two dimensions most directly aligned with the objective formulation of the proposed NAS framework: semantic generation quality and deployment-oriented efficiency. Semantic quality is quantified through the METEOR score, which is also the quality objective optimized during search. This preserves consistency between the optimization stage and the final post-NAS evaluation, allowing the selected architectures to be assessed directly with respect to the criterion that guided the search.

In addition to METEOR, the evaluation pipeline records hardware-oriented indicators during execution, namely power consumption, memory usage, GPU utilization, and CPU frequency. These measurements characterize the practical behavior of the evaluated architectures under deployment-oriented conditions and make it possible to determine whether semantic improvements are accompanied by favorable device-level efficiency profiles.

By jointly considering METEOR and hardware-related indicators, the framework enables a direct analysis of the trade-offs recovered by the NAS process. In particular, this evaluation allows us to determine whether the discovered architectures improve upon the baseline only in terms of the target semantic metric, or whether they also provide more attractive deployment characteristics for resource-constrained scenarios.

3.4.2. Pipeline Overview

The overall workflow of the proposed framework is illustrated in Figure 2. The process begins with the training and evaluation of the DistilGPT2 baseline, from which both METEOR and hardware-related indicators are collected. The NAS process is then executed over the proposed search space using the model-based evaluation mechanism described above, yielding candidate architectures that satisfy the search objectives. The selected architectures are subsequently trained and re-evaluated under the same protocol as the baseline to enable a direct final comparison.

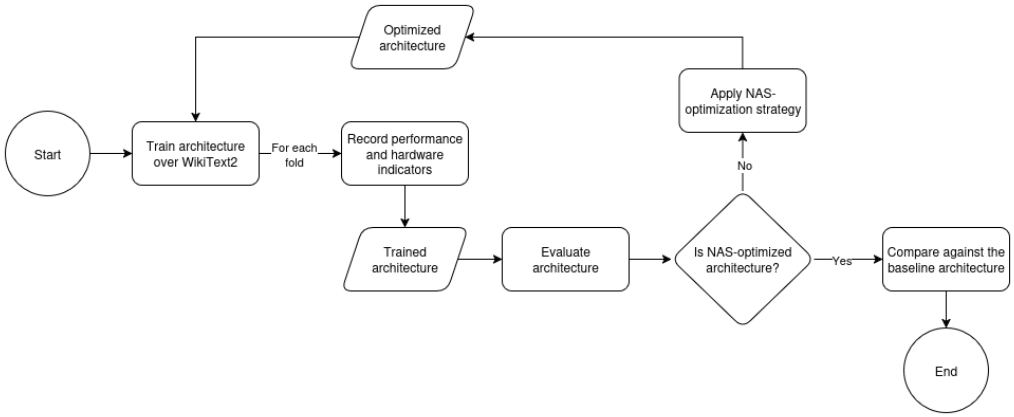

Figure 2: Overview of the evaluation pipeline used to compare the DistilGPT2 baseline and the NAS-discovered architectures.

Conceptually, the workflow is divided into two main components: the baseline evaluation stage and the NAS stage. During training and inference, all relevant indicators are recorded for subsequent analysis. After training, each architecture is evaluated on the test split under the same protocol, ensuring methodological consistency across the baseline and the NAS-selected models.

3.5. Experimental Setup

For the initial evaluation phase, WikiText2 is preprocessed to ensure compatibility with the selected baseline architecture. The preprocessing pipeline consists of two stages: texts are first grouped according to their length to support stratified data handling during training, and the resulting dataset is then tokenized using the DistilGPT2 tokenizer [51, 52].

Training follows a fixed configuration across all evaluated architectures. In particular, 500 warm-up steps are used to stabilize optimization during the early stages of training [53, 54, 55], and a weight decay of 0.01 is applied to improve generalization and reduce overfitting [56, 57, 58]. The same training protocol is used for both the baseline and the NAS-selected architectures in order to preserve consistency across evaluations.

For final evaluation, each sample in the original test split is modified by removing the last 30% of its text, which is treated as the target continuation. The remaining prefix is then provided as input to the architecture, and the generated continuation is compared against the original held-out segment.

3.5.1. NAS Search Configurations

Table 3 summarizes the NAS configurations considered for SMS-EMOA and Lex-MODES. Both search strategies were evaluated under four settings differing in population size, generation budget, and number of repetitions. The first two configurations serve as lightweight exploratory regimes, whereas the remaining ones increase the search budget to encourage broader exploration of the search space.

Table 3: NAS configurations evaluated for SMS-EMOA and Lex-MODES.

Algorithm	**Config.**	**# Exps.**	**Gens.**	**Self-adapt.**	F	μ
SMS-EMOA	1	30	100	–	–	10
	2	30	100	–	–	100
	3	20	250–500	–	–	200
	4	10	250–500	–	–	400
Lex-MODES	1	30	100	No	2.0	10
	2	30	100	Yes	0.5	100
	3	20	250–500	Yes	0.5	20
	4	10	250–500	Yes	2.0	400

For configurations with larger search budgets, an early convergence criterion is incorporated to avoid unnecessary evaluations. This mechanism monitors the average METEOR score across generations and terminates the search when the population remains within a narrow high-performance band, indicating limited recent improvement. Although this strategy reduces computational cost, it may also increase the risk of premature convergence. In addition, Configurations 3 and 4 include one extended run of 500 generations

for Pareto-front visualization, whereas the standard search budget used in the main experiments is 250 generations.

4. Results

This section first analyzes the DistilGPT2 baseline in order to establish a reference point for subsequent comparison, and then presents the architectures recovered by the NAS process.

4.1. Baseline Checkpoint Selection

Before comparing the NAS-generated architectures against the reference model, it is necessary to identify a DistilGPT2 checkpoint that is consistent with the optimization perspective adopted in this study. Since the proposed framework is centered on semantic quality, quantified through METEOR, and deployment-oriented behavior, characterized through hardware-related indicators, the baseline selection should be guided by these same criteria.

Table 4 summarizes the average METEOR score and hardware consumption across DistilGPT2 checkpoints trained for increasing numbers of epochs. From the semantic perspective, the 3-epoch checkpoint achieves the highest METEOR value, reaching 0.6961. Beyond this point, METEOR decreases slightly but consistently, down to 0.6932 at 15 epochs. Although the magnitude of this reduction is modest, the trend indicates that longer training does not improve the target quality criterion used throughout the NAS process.

Table 4: METEOR score and hardware comparison of DistilGPT2 checkpoints across training epochs.

Epochs	**METEOR**	**Power (mW)**	**Memory (kB)**	**GPU (%)**	**CPU (kHz)**
3	**0.6961**	1,671.56	54,269.24	21.41	84,218.43
5	0.6950	1,493.90	326.52	14.88	75,496.12
10	0.6941	1,428.90	568.20	**13.88**	91,772.40
12	0.6947	**1,391.25**	**247.87**	14.85	**50,376.76**
15	0.6932	1,408.14	515.42	14.74	90,131.32

From the hardware perspective, the results do not follow a strictly monotonic pattern. Later checkpoints improve specific indicators: the 12-epoch model yields the lowest power consumption, memory usage, and CPU frequency, whereas the 10-epoch model attains the lowest GPU utilization.

However, these advantages are dispersed across different checkpoints rather than concentrated in a single clearly dominant alternative.

To determine whether these differences were meaningful, a Wilcoxon signed-rank test was applied to power consumption, memory usage, GPU load, and CPU frequency. The resulting p-values ranged from 0.2 to 0.6, indicating that none of the observed differences was statistically significant ($p > 0.05$). Thus, despite visible variation in the average values, the inference-time hardware profiles remain broadly comparable across checkpoints.

Taken together, these results justify selecting the 3-epoch DistilGPT2 checkpoint as the reference baseline for post-NAS evaluation. Although it is not the best in every individual hardware indicator, it achieves the highest METEOR score, incurs the lowest training cost, and is not statistically outperformed in hardware behavior by later checkpoints. Under the joint perspective of semantic quality and deployment-oriented efficiency adopted in this work, the 3-epoch checkpoint therefore provides the most balanced and methodologically consistent baseline for comparison.

4.2. Configuration-level Characterization and Selection of Post-NAS Models

Table 5 summarizes the best architecture recovered in each NAS configuration. The resulting characterization reveals a clear contrast between the behaviors of SMS-EMOA and Lex-MODES.

SMS-EMOA remained largely confined to a compact basin across its four configurations. The dominant structural core changed little from one configuration to another, and most of the residual variation was concentrated in dropout refinement and, less frequently, in the number of attention heads. Thus, rather than inducing clearly distinct architectural regimes, the SMS-EMOA configurations mainly explored different regularized variants within the same low-complexity region of the search space. Despite their structural simplicity, these compact solutions still achieved competitive METEOR values, highlighting the ability of SMS-EMOA to recover efficient trade-offs under strong complexity pressure.

By contrast, Lex-MODES exhibited specialized architectural exploration, with two dominant search regimes emerging across configurations. Configurations 1 and 2 repeatedly converged to a compact attractor centered on $[8, 512, 48, 4, 0, 0, 0]$, whereas Configurations 3 and 4 shifted toward a recurrent high-capacity attractor centered on $[128, 512, 48, 64, 0, 0.30, 0]$. This indicates that increasing the search budget in Lex-MODES mainly changed the

Table 5: Configuration-level characterization of the best architectures recovered by each NAS setting. Architectures are encoded as $[e, l, n, h, r, a, g]$.

Algorithm	**Cfg.**	**Best encoded architecture**	**Characterization**
Lex-MODES	1	[8, 512, 48, 4, 0, 0, 0]	Aggressive exploration combining extremely compact solutions with occasional larger high-performing alternatives; repeated convergence to the same parameter-efficient architecture.
Lex-MODES	2	[8, 512, 48, 4, 0, 0, 0]	Broader exploration than Configuration 1, but the same compact trade-off remained dominant.
Lex-MODES	3	[128, 512, 48, 64, 0, 0.30, 0]	Shift toward a high-capacity, high-METEOR solution with stronger convergence around a single dominant architecture.
Lex-MODES	4	[128, 512, 48, 64, 0, 0.30, 0]	Stable convergence to the same high-capacity attractor found in Configuration 3, with only minor variation beyond dropout refinement.
SMS-EMOA	1	[8, 512, 2, 4, 0, 0, 0]	Predominantly compact models with low structural diversity; some regularized low-cost solutions still achieved acceptable METEOR values.
SMS-EMOA	2	[8, 512, 2, 8, 0, 0.30, 0.20]	Convergence to very compact architectures with moderate-to-high dropout; variability was concentrated on regularization rather than structure.
SMS-EMOA	3	[8, 512, 2, 8, 0, 0, 0.20]	Highly homogeneous compact regime in which optimization focused almost exclusively on dropout refinement.
SMS-EMOA	4	[8, 512, 2, 4, 0, 0.30, 0.20]	Same compact basin with stronger dropout tuning; the architecture remained effectively fixed.

dominant trade-off regime from a compact solution to a high-capacity, high-METEOR solution. Overall, this broader exploration enabled Lex-MODES to identify architectures with larger embedding sizes and deeper structures, while still showing repeated convergence to a small number of dominant configurations.

The configuration-level characterization indicates that the search process was highly deterministic and exploitative. Although eight configurations were evaluated, only six unique best architectures were recovered, with repeated convergence to identical or near-identical solutions. This limited diversity suggests that the current NAS setup explores a relatively narrow subset of the search space. Possible causes include the absence of explicit diversity-preservation mechanisms, a potentially restrictive hypervolume reference-point design, and an architectural encoding that may bias the

search toward specific model families. In the particular case of Lex-MODES, the lexicographic selection pressure may further reinforce convergence toward compact yet familiar solutions by prioritizing parameter efficiency whenever trade-off differences are small.

Although six unique best architectures were identified overall, only four were retained for the final post-NAS comparison against the DistilGPT2 baseline. This reduction was not intended to disregard the remaining two unique solutions, but to preserve a final evaluation that was both representative and methodologically consistent. Specifically, the retained models satisfy two conditions: first, they capture the main architectural regimes recovered by the search, namely a recurrent compact Lex-MODES solution, a recurrent high-capacity Lex-MODES solution, a representative compact SMS-EMOA solution, and an intermediate-complexity compromise solution; second, they are the candidates for which the complete post-search evaluation pipeline was available, including hardware-related measurements.

Importantly, the fourth retained model should not be interpreted as the best architecture of a single configuration. Instead, it was included as an additional intermediate Lex-MODES candidate selected for final evaluation in order to represent a mid-range trade-off regime between the recurrent compact and high-capacity attractors. By comparison, the remaining unique solutions were mainly compact SMS-EMOA variants that differed primarily in dropout tuning or in the number of attention heads, and therefore added limited structural diversity to the final comparison. Under this criterion, the final set should be understood not as an arbitrary reduction of the six unique solutions, but as the subset that preserves the principal trade-off regimes while enabling a uniform comparison in terms of semantic quality and deployment-oriented behavior.

Table 6 summarizes the models retained for final evaluation. In addition to the four NAS-selected architectures, the DistilGPT2 baseline is included as the reference architecture for direct comparison.

Together, the DistilGPT2 baseline and these four NAS-selected models provide a compact yet representative basis for the final comparison in terms of semantic quality and deployment-oriented behavior.

4.3. Post-NAS Evaluation

Based on the baseline analysis, the 3-epoch DistilGPT2 checkpoint was retained as the reference model for post-NAS evaluation. Since the proposed

Table 6: Models retained for final evaluation, including the DistilGPT2 baseline reference.

Model	**Algorithm / source**	**Encoded architecture**	**Role in the final comparison**
Baseline	DistilGPT2	[768, 1024, 6, 12, 0.10, 0.10, 0]	Reference architecture used as the primary baseline for post-NAS comparison.
Model 1	Lex-MODES	[8, 512, 48, 4, 0, 0, 0]	Recurrent compact solution recovered by Configurations 1 and 2.
Model 2	Lex-MODES	[128, 512, 48, 64, 0, 0.30, 0]	Recurrent high-capacity, high-METEOR solution recovered by Configurations 3 and 4.
Model 3	SMS-EMOA	[8, 512, 2, 4, 0.01, 0.30, 0.20]	Representative lightweight solution summarizing the compact regime repeatedly favored by SMS-EMOA.
Model 4	Lex-MODES	[64, 512, 48, 32, 0, 0.20, 0.30]	Additional intermediate-complexity candidate retained to represent a mid-range trade-off regime; unlike Models 1 and 2, it was not the best architecture of a single configuration.

Table 7: Meteore Score and hardware measurements consumption for NAS-generated models on the Jetson Orin Nano; in bold the statistics with the lower consumption and in italics the ones with higher consumption.

Architecture	**METEOR**	**Power (mW)**	**Memory (kB)**	**GPU (%)**	**CPU (kHz)**
DistilGPT2 (3 ep.)	0.6961	1,671.56	54,269.24	21.41	84,218.43
Model 1	**0.7200**	449.10	268,499.98	20.36	189,263.54
Model 2	0.7000	**395.95**	**2,535.00**	**1.55**	**4,000.00**
Model 3	0.7000	510.42	20,167.29	2.87	60,381.53
Model 4	0.7000	412.58	22,184.68	14.86	63,605.43

NAS framework is centered on semantic quality and efficiency, the comparison is restricted to METEOR and the hardware-related indicators collected during inference.

Table 7 summarizes the joint comparison between the selected DistilGPT2 baseline and the four NAS-generated architectures. From the semantic perspective, all NAS-generated models improve upon the baseline in METEOR, increasing from 0.6961 to values between 0.7000 and 0.7200. This confirms that the search process was effective with respect to its primary optimization target. In particular, Model 1 achieved the highest METEOR value, indicating that the search was able to recover at least one architecture that improves semantic generation quality beyond the selected baseline.

However, the hardware-oriented analysis shows that semantic improve-

ment and deployment efficiency were not fully aligned. Model 1, despite achieving the highest METEOR value, exhibited the least favorable runtime profile among the NAS-generated models in terms of memory usage and CPU demand, even exceeding the baseline in both indicators. This result is important because it shows that optimizing the target semantic metric does not necessarily produce architectures that are attractive for deployment.

By contrast, Model 2 represents the strongest hardware-oriented outcome of the search. Although its METEOR score was not the highest among the post-NAS candidates, it attained the lowest values across all recorded hardware indicators, including power consumption, memory usage, GPU utilization, and CPU frequency. Model 2 therefore provides the clearest evidence that the proposed framework can recover highly efficient deployment-oriented candidates while remaining competitive in semantic quality.

Models 3 and 4 occupy intermediate regions of the trade-off surface. Model 3 improves upon the baseline in all hardware-related indicators while maintaining a METEOR value above the baseline, but remains less efficient than Model 2. Model 4 also improves upon the baseline across all hardware indicators and preserves a competitive METEOR value, thereby representing the most balanced compromise among the NAS-generated alternatives. Although it does not match the extreme hardware efficiency of Model 2 or the peak semantic performance of Model 1, it offers the most even trade-off between semantic quality and deployment-oriented behavior.

Taken together, these results support two main conclusions. First, the NAS process is effective with respect to its target semantic objective, since all selected architectures improve upon the baseline in METEOR. Second, the framework is capable of identifying architectures with substantially improved hardware profiles. Nevertheless, the contrast between Models 1 and 2–4 also exposes a structural limitation of the current formulation: parameter count alone is not sufficient to characterize real deployment efficiency. Architectures that are competitive in the bi-objective search space may still differ substantially in memory footprint, processor demand, and overall device-level behavior due to differences in architectural organization and implementation-level effects.

From a deployment-oriented NAS perspective, this is arguably the main finding of the post-NAS evaluation. The current formulation is sufficient to recover meaningful semantic–efficiency trade-offs, but it does not guarantee that the architectures favored during search will be consistently deployment-optimal. This suggests that future versions of the framework should incor-

porate explicit hardware-aware objectives, such as latency, memory usage, or energy-related indicators, directly into the optimization loop rather than relying exclusively on parameter count as a practical proxy for efficiency.

5. Conclusions and Future Work

This work investigated the feasibility of applying NAS to text generation through a DistilGPT2-based search space, multi-objective evolutionary search, and model-based performance prediction. The results show that NAS can be effectively adapted to text generation and can recover meaningful trade-offs between semantic quality and deployment-oriented efficiency. In particular, all selected post-NAS architectures improved upon the DistilGPT2 baseline in terms of METEOR, while some candidates also exhibited substantially improved hardware profiles.

At the same time, the results exposed two main limitations. First, semantic gains were not consistently aligned with the most favorable hardware behavior, indicating that parameter count alone is not sufficient to characterize real deployment efficiency. Second, despite evaluating eight configurations, only six unique best architectures were recovered, revealing limited diversity and repeated convergence to a small number of attractors. This suggests that the current framework still explores a relatively narrow portion of the search space, likely due to the absence of explicit diversity-preservation mechanisms, the current hypervolume reference-point design, and the limited expressiveness of the architectural encoding.

Future work should therefore proceed in two directions. First, the NAS formulation should be strengthened through richer architectural encodings, stronger diversity-promotion mechanisms, and explicit hardware-aware objectives such as latency, memory footprint, and energy consumption. Second, the framework should move beyond architecture-only optimization and incorporate state-of-the-art deployment-oriented techniques such as pruning, post-training quantization, quantization-aware training, mixed-precision policies, and knowledge distillation. Such extensions would enable a tighter coupling between architecture search and real hardware constraints, leading to more robust and practically deployable text generation models. More broadly, validation across additional datasets, tasks, and evaluation protocols will be necessary to assess the generality of the recovered trade-offs.

References

[1] A. Thessen, H. Cui, D. Mozzherin, Applications of Natural Language Processing in Biodiversity Science, Advances in Bioinformatics (2012).

[2] O. Uzuner, A. Stubbs, Practical Applications for Natural Language Processing in Clinical Research, Department of Health and Human Services USA (2015).

[3] Y. Ding, J. Ma, X. Luo, Applications of Natural Language Processing in construction, Automation in construction (2022).

[4] K. Church, L. Rau, Commercial Applications of Natural Language Processing, Advances in Traditional AI (1995).

[5] A. Montejo-Ráez, S. Jiménez-Zafra, Current Approaches and Applications in Natural Language Processing, Applied Sciences (2022).

[6] J. Xu, Z. Li, W. Chen, Q. Wang, X. Gao, Q. Cai, Z. Ling, On-Device Language Models: A Comprehensive Review, arXiv Preprint (2024).

[7] A. Ramachandran, Empowering Edge AI with Small Language Models Architectures, Challenges and Transformative Enterprise Applications, arXiv Preprint (2024).

[8] Y. Liu, Y. Sun, B. Xue, M. Zhang, G. Yen, K. Tan, A Survey on Evolutionary Neural Architecture Search, IEEE Transactions on Neural Networks and Learning Systems (2023) 550–570.

[9] Y. Wang, Y. Yang, Y. Chen, J. Bai, C. Zhang, G. Su, X. Kou, Y. Tong, M. Yang, L. Zhou, TextNAS: A Neural Architecture Search Space Tailored for Text Representation, The Thirty-Fourth AAAI Conference on Artificial Intelligence (2020).

[10] C. Yu, X. Liu, Y. Wang, Y. Liu, W. Feng, D. Xiong, C. Tang, J. Lv, GPT-NAS: Evolutionary Neural Architecture Search with the Generative Pre-Trained Model, arXiv Preprint (2025).

[11] S. Schröder, B. Hammer, Neural Architecture Search for Sentence Classification with BERT, arXiv Preprint (2024).

[12] T. Elksen, F. Hendrik, J.and Hutter, Neural Architecture Search: A survey, Journal or Machine Learning Research (2019).

[13] K. Teja, M. Emani, V. Vishwanath, A. Somani, Neural Architecture Search for Transformers: A survey, IEEE Access (2022).

[14] H. Benmezaine, K. El Maghraoui, H. Ouarnoughi, S. Niar, M. Witsuba, N. Wang, A comprehensive survey on Hardware-Aware Neural Architecture Search, IEEE (2021).

[15] C. White, A. Zela, R. Ru, Y. Liu, F. Hutter, How powerful are performance predictors in neural architecture search?, Advances in neural information processing systems 34 (2021) 28454–28469.

[16] L. Kinde, Testing AI-democratization: What are the lower limits of textgeneration using artificial neural networks?, Luleå University of Technology Department of Computer Science, Electrical and Space Engineering (2019).

[17] P. Ren, Y. Xiao, X. Chang, P. Huang, Z. Li, X. Chen, X. Wang, A Comprehensive Survey of Neural Architecture Search: Challenges and Solutions, ACM Comput. Surv. 54 (2021).

[18] S. Binitha, S. Sathya, A Survey of Bio inspired Optimization Algorithms, International Journal of Soft Computing and Engineering (IJSCE) (2012).

[19] A. Hamid, B. Gyires-Tóth, International Conference on Learning Representations, IEEE (2021).

[20] S. Reddy, A. Tyagi, J. Mundra, S. Mukherjee, S. Raj, A. Garimella, N. Chhaya, AutoSumm: Automatic Model Creation for Text Summarization, Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (2021).

[21] J. Gou, B. Yu, S. Maybank, D. Tao, Knowledge Distillation: A Survey, International Journey of Computer Vision (2021).

[22] L. Kurisinkel, N. Chen, Set to Ordered Text: Generating discharge instructions from medical billing codes, Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (2019).

[23] Z. Lu, I. Whalen, V. Boddeti, Y. Dhebar, K. Deb, E. Goodman, W. Banzhaf, Nsga-net: neural architecture search using multi-objective genetic algorithm, in: Proceedings of the genetic and evolutionary computation conference, 2019, pp. 419–427.

[24] J.-D. Dong, A.-C. Cheng, D.-C. Juan, W. Wei, M. Sun, Dpp-net: Device-aware progressive search for pareto-optimal neural architectures, in: Proceedings of the European conference on computer vision (ECCV), 2018, pp. 517–531.

[25] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, Q. V. Le, Mnasnet: Platform-aware neural architecture search for mobile, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2019, pp. 2820–2828.

[26] H. Cai, L. Zhu, S. Han, Proxylessnas: Direct neural architecture search on target task and hardware, arXiv preprint arXiv:1812.00332 (2018).

[27] B. Wu, X. Dai, P. Zhang, Y. Wang, F. Sun, Y. Wu, Y. Tian, P. Vajda, Y. Jia, K. Keutzer, Fbnet: Hardware-aware efficient convnet design via differentiable neural architecture search, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2019, pp. 10734–10742.

[28] N. Sinha, P. Rostami, A. El Rahman Shabayek, A. Kacem, D. Aouada, Multi-objective hardware aware neural architecture search using hardware cost diversity, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 8032–8039.

[29] N. N. Wulandari, S. A. I. Alfarozi, I. Ardiyanto, Lightweight swin-transformer and distilgpt2 for image-to-latex in constrained environments, in: 2025 International Conference on Advanced Technologies in Energy and Informatic (ICATEI), IEEE, 2025, pp. 73–78.

[30] V. Sanh, L. Debut, J. Chaumond, T. Wolf, DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter, arXiv Preprint (2020).

[31] J. Moody, The Effective Number of Parameters: An Analysis of Generalization and Regularization in Nonlinear Learning Systems, Advances in neural information processing systems 4 (1991).

[32] D. Justus, J. Brennan, S. Bonner, A. McGough, Predicting the Computational Cost of Deep Learning Models, 2018 IEEE international conference on big data (Big Data) (2018) 3873–3882.

[33] R. Dwivedi, C. Singh, B. Yu, M. Wainwright, Revisiting Minimum Description Length Complexity in Overparameterized Models, Journal of Machine Learning Research 24 (2023) 1–59.

[34] J. Kaplan, S. McCandlish, T. Henighan, T. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, D. Amodei, Scaling Laws for Neural Language Models, arXiv Preprint (2020).

[35] L. Verma, K. Kolhatkar, Optimizing transformer-based machine translation model for single GPU training: A hyperparameter ablation study, arXiv Preprint (2023).

[36] N. Beume, B. Naujoks, M. Emmerich, SMS-EMOA: Multiobjective selection based on dominated hypervolume, Elsevier (2006).

[37] W. Zheng, B. Doerr, Runtime Analysis of the SMS-EMOA for Many-Objective Optimization, Proceedings of the AAAI Conference on Artificial Intelligence 38 (2024) 20874–20882.

[38] N. Beume, M. Laumanns, G. Rudolph, Convergence rates of SMS-EMOA on continuous bi-objective problem classes, Proceedings of the 11th workshop proceedings on Foundations of genetic algorithms (2011).

[39] I. Ali, A Differential Evolution Algorithm for Generic Combinatorial Optimization Problems, Ph.D. thesis, University of New South Wales at Canberra, Australia, 2020.

[40] N. Awad, N. Mallik, F. Hutter, Differential Evolution for Neural Architecture Search, 1st Workshop on Neural Architecture Search at ICLR (2020).

[41] Z. Kuş, B. Kiraz, T. Koçak, M. Aydın, E. Özkan, A. Vural, A. Kiraz, B. Can, Differential Evolution-based Neural Architecture Search for brain vessel segmentation, Engineering Science and Technology, an International Journal (2023).

[42] J. Lampien, I. Zelinka, Mixed Integer-Discrete-Continuous Optimization by Differential Evolution, Proceedings of MENDEL'99, 5th International Mendel Conference on Soft Computing (1999).

[43] J. Cai, G. Thierauf, Evolution strategies for solving discrete optimization problems, Elsevier (1995).

[44] B. Doerr, F. Neumann, A Survey on Recent Progress in the Theory of Evolutionary Algorithms for Discrete Optimization, ACM Transactions on Evolutionary Learning and Optimization (2021).

[45] I. Rechenberg, The Evolution Strategy. A Mathematical Model of Darwinian Evolution, Springer Series in Synergetics (1984).

[46] L. Khilkova, Cross-Selection Based Evolution Strategies, Internatinal Journal of Computing (2023).

[47] S. Sarmiento, J. Llano, J. Falcón, R. Monroy, M. Casillas, V. Sosa, Surrogate Modeling for Efficient Evolutionary Multi-Objective Neural Architecture Search in Super Resolution Image Restoration, Scite Press (2024).

[48] S. Sarmiento-Rosales, V. A. S. Hernández, R. Monroy, Evolutionary neural architecture search for super-resolution: Benchmarking synflow and model-based predictors, Swarm and Evolutionary Computation 100 (2026) 102236.

[49] S. Merity, C. Xiong, J. Bradbury, R. Socher, Pointer Sentinel Mixture Models, arXiv Preprint (2016).

[50] W.-L. Loh, On latin hypercube sampling, The annals of statistics 24 (1996) 2058–2080.

[51] A. Tacorda, M. Ignacio, N. Oco, R. Roxas, Controlling Byte-Pair Encoding for Neural Machine Translation, 2017 International Conference on Asian Language Processing (IALP) (2017) 168–171.

[52] A. Vaswani, N. Shazeer, P. N., J. Uszkoreit, L. Jones, A. Gomez, L. Kaiser, I. Polosukhin, Attention Is All You Need, CoRR (2017).

[53] J. Gilmer, B. Ghorbani, A. Garg, S. Kudugunta, B. Neyshabur, D. Cardoze, G. Dahl, Z. Nado, O. Firat, A Loss Curvature Perpsective on Training Instability in Deep Learning, ICLR (2022).

[54] K. Nakamura, B. Derbe, K. Won, B. Hong, Learning-Rate Annealing Methods for Deep Neural Networks, Electronics (2021).

[55] Y. Zhu, R. Xie, F. Zhuang, K. Ge, Y. Sun, X. Zhang, L. Lin, J. Cao, Learning to Warm Up Cold Item Embeddings for Cold-start Recommendation with Meta Scaling and Shifting Networks, SIGIR (2021).

[56] A. Kroght, J. Hertz, A Simple Weight Decay Can Improve Generalization, Advances in Neural Information Processing Systems (1991).

[57] I. Loshchilov, F. Hutter, Decoupled Weight Decay Regularization, ICLR (2019).

[58] Z. Xie, I. Sato, M. Sugiyama, Stable Weight Decay Regularization, ICLR (2021).